**CSCI 4390 Senior Project**
**Project Proposal**

**Title**: Optimizing CPU/GPU Coordination for Efficient Classification Training

**Team Members**:
Kayla Garza (kayla.garza08@utrgv.edu)
Jacob Garza (jacob.garza12@utrgv.edu)
Juan Amaya (juan.amayahernandez01@utrgv.edu)
Esmer Gomez (esmeralda.gomezninoderivera01@utrgv.edu)

**Faculty Adviser**:
Dr. Bin Fu (bin.fu@utrgv.edu)
Jose Nunez (UTRGV Graduate Student - jose.nunez01@utrgv.edu)

**Problem/Opportunity Statement**: We propose to improve the CPU/GPU coordination and apply parallel computing techniques to speed up Bayesian hyperparameter optimization and overall AI training for image classification models.

**Background**: Image classification models, especially deep learning models like Convolutional Neural Networks (CNNs), require a lot of computing power to train. These models are trained using gradient descent methods, which update model weights step by step to minimize error. Gradient descent and its optimizers such as stochastic gradient descent (SGD) and Adam are the foundation of training in many AI systems. GPUs are commonly used to speed up this process because they can handle many calculations at the same time. However, training performance still depends on how well the CPU and GPU work together. The CPU usually handles tasks like preparing data, managing memory, and scheduling jobs, while the GPU performs the mathematical computations. When Bayesian optimization is used to search for the best hyperparameters, the model must be trained many times with different settings. These repeated training runs can be slow if CPU/GPU coordination is inefficient. Poor scheduling, slow data transfers, or uneven workloads can decrease performance and limit the benefits of using a GPU.

**Survey of Solutions:** This problem has been studied before, and there are several existing approaches to improve the speed of hyperparameter optimization and AI training. First, traditional Bayesian optimization runs one hyperparameter configuration at a time. Each configuration trains the model and then updates the optimization process before selecting the next set of parameters. This method is easy to implement and stable, but it is slow because the training runs are done sequentially and don't fully use available CPU and GPU resources. Second, distributed hyperparameter tuning frameworks allow multiple hyperparameter trials to run in parallel across CPUs or multiple GPUs. These systems have built-in parallelism, can scale across machines, and are widely used in research. However, they are often designed for large distributed systems and can introduce extra overhead or complexity. Third, some approaches rely only on GPU acceleration for faster training. While GPUs speed up computations, they don't automatically improve CPU scheduling or efficiently manage multiple concurrent hyperparameter trials. Our proposed solution focuses on improving CPU/GPU coordination during Bayesian hyperparameter optimization, rather than only relying on existing distributed tools. We aim to reduce idle time and improve overall system efficiency within the training pipeline.

**Proposed Solution**: In this project we will design a parallel computing framework to speed up Bayesian hyperparameter optimization by improving coordination between the CPU and GPU. Instead of running each hyperparameter one at a time, we will run multiple independent training experiments in parallel.

The CPU will handle scheduling tasks, loading data, and managing the optimization process, while the GPU will perform the model training computations. Since each training run uses gradient descent methods to update the model's weights, by running multiple training processes at the same time, we can make this optimization step faster and efficient. We will explore methods such as running concurrent training jobs on smaller subsets of data, improving task scheduling and recuing idle GPU time to make better use of system resources.

We will start with an existing sequential implementation of the first stage of Bayesian optimization and revise it for parallel execution. To measure improvement, we will compare training time, GPU usage, and overall throughput between the original version and the optimized parallel version. The project will be considered successful if we significantly reduce hyperparameter search time while maintaining model performance and accuracy.

**Survey of Technology:** To develop our solution, we plan to use PyTorch as our deep learning framework to build and train the image classification models. For hyperparameter tuning, we will use a Bayesian optimization library like Optuna. We will rely on NVIDIA GPUs with CUDA to accelerate model training, and we will use Python as our main programming language because of its strong support for machine learning and parallel computing tools.

**Risk Considerations:** One major challenge of this project is managing parallel execution without causing resource conflicts or memory issues. Running multiple training jobs at the same time could use too much GPU memory or lower performance if not scheduled properly. Another challenge is making sure that parallel execution doesn't affect the accuracy of the Bayesian optimization process.

**Project Management Strategy:** We plan to manage this project through clear and consistent communication. It is important that all team members understand their tasks and stay on track so we can complete the project on time. If a team member faces issues, we will support each other to make sure everyone succeeds and the project continues moving forward. We coordinate regularly through our Discord group chat to share updates and ask questions. In addition, we hold weekly meetings with our advisor to review our progress, discuss any issues, and make sure we are all on the same page.